* Introduction to HTML and CSS:

HTML (Hypertext Markup Language): is the standard markup language used to create and structure content on webpages. It defines the basic structure of webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multime-dia content. HTML documents are interepted by web browsers to display the content in a structured and meaningful manner. HTML provides structure and semantic organization of a webpage.

CSS (cascading style sheets):

It is a style sheet language used to control the presentation & visual appearance of HTML Documents. It is used to define properties such as colors, fonts, background, spacing, borders, alignment, dimensions and layout. CSS also supports responsive design, allowing webpages to adapt to different screen sized and devices CSS can be applied used inline, internal or external. HTML & CSS work together to develop

* HTML ಎಂದರೆ ಏನು? ಅದರ ವಿಸ್ತೃತ ರೂಪವನ್ನು ಬರೆಯಿರಿ.

HTML ಎಂದರೆ HyperText Markup Language. ಇದು ವೆಬ್‌ಪುಟಗಳನ್ನು ರಚಿಸಲು ಬಳಸುವ ಪ್ರಮಾಣಿತ ಭಾಷೆ.

* ಕೆಳಗಿನ ಪದಗಳ ಅರ್ಥವನ್ನು ಬರೆಯಿರಿ.

ಪ್ಲಾಟ್‌ಫಾರ್ಮ್ ಎಂದರೆ ಕಂಪ್ಯೂಟರ್ ಅಥವಾ ಸಾಫ್ಟ್‌ವೇರ್ ಚಲಿಸುವ ಆಧಾರ ವ್ಯವಸ್ಥೆ.

ಸಾಫ್ಟ್‌ವೇರ್ ಎಂದರೆ ಕಂಪ್ಯೂಟರ್‌ಗೆ ಸೂಚನೆಗಳನ್ನು ನೀಡುವ ಪ್ರೋಗ್ರಾಂಗಳ ಸಮೂಹ.

ಹಾರ್ಡ್‌ವೇರ್ ಎಂದರೆ ಕಂಪ್ಯೂಟರ್‌ನ ಭೌತಿಕ ಭಾಗಗಳು.

ಪ್ರೋಗ್ರಾಮಿಂಗ್ ಭಾಷೆ ಎಂದರೆ ಕಂಪ್ಯೂಟರ್‌ಗೆ ಆಜ್ಞೆಗಳನ್ನು ನೀಡಲು ಬಳಸುವ ಭಾಷೆ.

ಕಂಪೈಲರ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಮ್‌ನ್ನು ಯಂತ್ರ ಭಾಷೆಗೆ ಪರಿವರ್ತಿಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಇಂಟರ್‌ಪ್ರಿಟರ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಮ್‌ನ್ನು ಸಾಲು ಸಾಲಾಗಿ ಓದಿ ಕಾರ್ಯಗತಗೊಳಿಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಅಸೆಂಬ್ಲರ್ ಎಂದರೆ ಅಸೆಂಬ್ಲಿ ಭಾಷೆಯನ್ನು ಯಂತ್ರ ಭಾಷೆಗೆ ಪರಿವರ್ತಿಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಲಿಂಕರ್ ಎಂದರೆ ಹಲವು ಆಬ್ಜೆಕ್ಟ್ ಫೈಲ್‌ಗಳನ್ನು ಒಂದೇ ಎಕ್ಸಿಕ್ಯೂಟೆಬಲ್ ಫೈಲ್ ಆಗಿ ಜೋಡಿಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಡೀಬಗ್ಗರ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಮ್‌ನಲ್ಲಿನ ದೋಷಗಳನ್ನು ಪತ್ತೆಹಚ್ಚಲು ಬಳಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಎಡಿಟರ್ ಎಂದರೆ ಪಠ್ಯವನ್ನು ಸಂಪಾದಿಸಲು ಬಳಸುವ ಸಾಫ್ಟ್‌ವೇರ್.

ಐಡಿಇ ಎಂದರೆ ಪ್ರೋಗ್ರಾಮಿಂಗ್‌ಗೆ ಅಗತ್ಯವಾದ ಎಲ್ಲ ಸಾಧನಗಳನ್ನು ಒಳಗೊಂಡ ಸಾಫ್ಟ್‌ವೇರ್.

ಎಪಿಐ ಎಂದರೆ ಸಾಫ್ಟ್‌ವೇರ್ ಘಟಕಗಳು ಪರಸ್ಪರ ಸಂವಹನ ನಡೆಸಲು ಬಳಸುವ ನಿಯಮಗಳ ಗುಂಪು.

ಎಸ್‌ಡಿಕೆ ಎಂದರೆ ಒಂದು ಪ್ಲಾಟ್‌ಫಾರ್ಮ್‌ಗಾಗಿ ಸಾಫ್ಟ್‌ವೇರ್ ರಚಿಸಲು ಬೇಕಾದ ಸಾಧನಗಳ ಸಂಗ್ರಹ.

ಫ್ರೇಮ್‌ವರ್ಕ್ ಎಂದರೆ ಅಪ್ಲಿಕೇಶನ್ ರಚನೆಗೆ ಮಾರ್ಗದರ್ಶನ ನೀಡುವ ಸಾಫ್ಟ್‌ವೇರ್ ರಚನೆ.

ಲೈಬ್ರರಿ ಎಂದರೆ ಪುನರ್ಬಳಕೆಗಾಗಿ ಸಂಗ್ರಹಿಸಿಟ್ಟ ಕೋಡ್‌ನ ಸಂಗ್ರಹ.

ಮಾಡ್ಯೂಲ್ ಎಂದರೆ ಸ್ವತಂತ್ರವಾಗಿ ಅಭಿವೃದ್ಧಿಪಡಿಸಬಹುದಾದ ಸಾಫ್ಟ್‌ವೇರ್‌ನ ಒಂದು ಭಾಗ.

ಪ್ಯಾಕೇಜ್ ಎಂದರೆ ಸಂಬಂಧಿತ ಮಾಡ್ಯೂಲ್‌ಗಳ ಸಂಗ್ರಹ.

ಕ್ಲಾಸ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ಗಳ ರಚನೆಗೆ ಬಳಸುವ ಟೆಂಪ್ಲೇಟ್.

ಆಬ್ಜೆಕ್ಟ್ ಎಂದರೆ ಕ್ಲಾಸ್‌ನ ಒಂದು ನಿದರ್ಶನ.

ಮೆಥಡ್ ಎಂದರೆ ಕ್ಲಾಸ್‌ನಲ್ಲಿ ವ್ಯಾಖ್ಯಾನಿಸಲಾದ ಕಾರ್ಯ.

ಅಟ್ರಿಬ್ಯೂಟ್ ಎಂದರೆ ಕ್ಲಾಸ್‌ನಲ್ಲಿ ವ್ಯಾಖ್ಯಾನಿಸಲಾದ ಡೇಟಾ.

ಇನ್‌ಹೆರಿಟೆನ್ಸ್ ಎಂದರೆ ಒಂದು ಕ್ಲಾಸ್‌ನಿಂದ ಇನ್ನೊಂದು ಕ್ಲಾಸ್‌ಗೆ ಗುಣಲಕ್ಷಣಗಳನ್ನು ಪಡೆಯುವುದು.

ಪಾಲಿಮಾರ್ಫಿಸಂ ಎಂದರೆ ಒಂದೇ ಹೆಸರಿನ ಮೆಥಡ್ ವಿಭಿನ್ನ ರೀತಿಯಲ್ಲಿ ವರ್ತಿಸುವುದು.

ಎನ್‌ಕ್ಯಾಪ್ಸುಲೇಷನ್ ಎಂದರೆ ಡೇಟಾ ಮತ್ತು ಮೆಥಡ್‌ಗಳನ್ನು ಒಟ್ಟಿಗೆ ಸೇರಿಸಿ ಮರೆಮಾಚುವುದು.

ಅಬ್‌ಸ್ಟ್ರಾಕ್ಷನ್ ಎಂದರೆ ಅನಗತ್ಯ ವಿವರಗಳನ್ನು ಮರೆಮಾಚಿ ಅಗತ್ಯ ಮಾಹಿತಿಯನ್ನು ಮಾತ್ರ ತೋರಿಸುವುದು.

ಕನ್‌ಸ್ಟ್ರಕ್ಟರ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್ ರಚನೆಯಾದಾಗ ಕರೆಯಲ್ಪಡುವ ವಿಶೇಷ ಮೆಥಡ್.

ಡೆಸ್ಟ್ರಕ್ಟರ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್ ನಾಶವಾಗುವಾಗ ಕರೆಯಲ್ಪಡುವ ವಿಶೇಷ ಮೆಥಡ್.

ಸ್ಟ್ಯಾಟಿಕ್ ಎಂದರೆ ಕ್ಲಾಸ್‌ಗೆ ಸೇರಿದ ಆದರೆ ಆಬ್ಜೆಕ್ಟ್‌ಗೆ ಅಲ್ಲದ ಸದಸ್ಯ.

ಫೈನಲ್ ಎಂದರೆ ಬದಲಾಯಿಸಲಾಗದ ಮೌಲ್ಯ ಅಥವಾ ಮೆಥಡ್.

ಅಬ್‌ಸ್ಟ್ರಾಕ್ಟ್ ಎಂದರೆ ಅಪೂರ್ಣ ಕ್ಲಾಸ್ ಅಥವಾ ಮೆಥಡ್.

ಇಂಟರ್‌ಫೇಸ್ ಎಂದರೆ ಕ್ಲಾಸ್‌ಗಳು ಅನುಸರಿಸಬೇಕಾದ ಒಪ್ಪಂದ.

ಎಕ್ಸೆಪ್ಷನ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂ ಚಾಲನೆಯಲ್ಲಿ ಉಂಟಾಗುವ ದೋಷ.

ಟ್ರೈ-ಕ್ಯಾಚ್ ಎಂದರೆ ಎಕ್ಸೆಪ್ಷನ್‌ಗಳನ್ನು ನಿರ್ವಹಿಸುವ ವಿಧಾನ.

ಥ್ರೋ ಎಂದರೆ ಎಕ್ಸೆಪ್ಷನ್ ಅನ್ನು ಉದ್ದೇಶಪೂರ್ವಕವಾಗಿ ಎಸೆಯುವುದು.

ಥ್ರೆಡ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂನ ಒಂದು ಚಾಲನಾ ಮಾರ್ಗ.

ಮಲ್ಟಿಥ್ರೆಡಿಂಗ್ ಎಂದರೆ ಹಲವು ಥ್ರೆಡ್‌ಗಳನ್ನು ಏಕಕಾಲದಲ್ಲಿ ಚಲಾಯಿಸುವುದು.

ಸಿಂಕ್ರೊನೈಸೇಷನ್ ಎಂದರೆ ಥ್ರೆಡ್‌ಗಳ ಸುರಕ್ಷಿತ ಪ್ರವೇಶ ನಿಯಂತ್ರಣ.

ಲಾಕ್ ಎಂದರೆ ಒಂದು ಸಮಯದಲ್ಲಿ ಒಂದು ಥ್ರೆಡ್ ಮಾತ್ರ ಪ್ರವೇಶಿಸಲು ಅನುಮತಿಸುವ ಯಾಂತ್ರಿಕತೆ.

ಡೆಡ್‌ಲಾಕ್ ಎಂದರೆ ಎರಡು ಥ್ರೆಡ್‌ಗಳು ಪರಸ್ಪರ ಕಾಯುತ್ತಿರುವ ಸ್ಥಿತಿ.

ಸ್ಟ್ರಿಂಗ್ ಎಂದರೆ ಅಕ್ಷರಗಳ ಸರಣಿ.

ಅರೇ ಎಂದರೆ ಒಂದೇ ಪ್ರಕಾರದ ಮೌಲ್ಯಗಳ ಸಂಗ್ರಹ.

ಲಿಸ್ಟ್ ಎಂದರೆ ಕ್ರಮಬದ್ಧ ಸಂಗ್ರಹ.

ಸೆಟ್ ಎಂದರೆ ಪುನರಾವರ್ತನೆಯಿಲ್ಲದ ಸಂಗ್ರಹ.

ಮ್ಯಾಪ್ ಎಂದರೆ ಕೀ-ಮೌಲ್ಯ ಜೋಡಿಗಳ ಸಂಗ್ರಹ.

ಸ್ಟ್ರೀಮ್ ಎಂದರೆ ಡೇಟಾದ ಹರಿವು.

ಫೈಲ್ ಎಂದರೆ ಶೇಖರಣೆಯಲ್ಲಿರುವ ಡೇಟಾ.

ಡೈರೆಕ್ಟರಿ ಎಂದರೆ ಫೈಲ್‌ಗಳ ಸಂಗ್ರಹ.

ಪಾತ್ ಎಂದರೆ ಫೈಲ್ ಅಥವಾ ಡೈರೆಕ್ಟರಿಯ ವಿಳಾಸ.

ಅಬ್ಸೊಲ್ಯೂಟ್ ಪಾತ್ ಎಂದರೆ ಪೂರ್ಣ ವಿಳಾಸ.

ರಿಲೇಟಿವ್ ಪಾತ್ ಎಂದರೆ ಪ್ರಸ್ತುತ ಸ್ಥಳಕ್ಕೆ ಸಂಬಂಧಿಸಿದ ವಿಳಾಸ.

ಕಮೆಂಟ್ ಎಂದರೆ ಕೋಡ್‌ನಲ್ಲಿ ಬರೆಯುವ ವಿವರಣೆ.

ಡಾಕ್ಯುಮೆಂಟೇಷನ್ ಎಂದರೆ ಸಾಫ್ಟ್‌ವೇರ್ ಬಗ್ಗೆ ಬರೆದ ಲಿಖಿತ ವಿವರಣೆ.

ಟೆಸ್ಟಿಂಗ್ ಎಂದರೆ ಸಾಫ್ಟ್‌ವೇರ್ ಸರಿಯಾಗಿ ಕೆಲಸ ಮಾಡುತ್ತಿದೆಯೇ ಎಂದು ಪರಿಶೀಲಿಸುವುದು.

ಯೂನಿಟ್ ಟೆಸ್ಟ್ ಎಂದರೆ ಒಂದು ಘಟಕವನ್ನು ಪ್ರತ್ಯೇಕವಾಗಿ ಪರೀಕ್ಷಿಸುವುದು.

ಇಂಟಿಗ್ರೇಷನ್ ಟೆಸ್ಟ್ ಎಂದರೆ ಹಲವು ಘಟಕಗಳನ್ನು ಒಟ್ಟಿಗೆ ಪರೀಕ್ಷಿಸುವುದು.

ರಿಗ್ರೆಷನ್ ಟೆಸ್ಟ್ ಎಂದರೆ ಬದಲಾವಣೆಗಳು ಹಳೆಯ ಕಾರ್ಯವನ್ನು ಮುರಿದಿಲ್ಲವೆಂದು ಪರೀಕ್ಷಿಸುವುದು.

ಡಿಬಗ್ ಎಂದರೆ ದೋಷಗಳನ್ನು ಪತ್ತೆಹಚ್ಚಿ ಸರಿಪಡಿಸುವುದು.

ಲಾಗ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂ ಚಾಲನೆಯ ದಾಖಲೆ.

ಕಾನ್ಫಿಗರೇಷನ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂನ ವರ್ತನೆಯನ್ನು ನಿಯಂತ್ರಿಸುವ ಸೆಟ್ಟಿಂಗ್‌ಗಳು.

ಎನ್‌ವಿರಾನ್‌ಮೆಂಟ್ ವೇರಿಯಬಲ್ ಎಂದರೆ ಆಪರೇಟಿಂಗ್ ಸಿಸ್ಟಮ್‌ನಲ್ಲಿ ಸಂಗ್ರಹಿಸಿಟ್ಟ ಮೌಲ್ಯ.

ಕಮಾಂಡ್ ಲೈನ್ ಆರ್ಗ್ಯುಮೆಂಟ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂ ಚಾಲನೆಯಾಗುವಾಗ ನೀಡುವ ಇನ್‌ಪುಟ್.

ಸ್ಟ್ಯಾಂಡರ್ಡ್ ಇನ್‌ಪುಟ್ ಎಂದರೆ ಕೀಬೋರ್ಡ್‌ನಿಂದ ಬರುವ ಡೇಟಾ.

ಸ್ಟ್ಯಾಂಡರ್ಡ್ ಔಟ್‌ಪುಟ್ ಎಂದರೆ ಸ್ಕ್ರೀನ್‌ಗೆ ಹೋಗುವ ಡೇಟಾ.

ಸ್ಟ್ಯಾಂಡರ್ಡ್ ಎರರ್ ಎಂದರೆ ದೋಷ ಸಂದೇಶಗಳಿಗಾಗಿ ಬಳಸುವ ಔಟ್‌ಪುಟ್.

ರೀಡೈರೆಕ್ಷನ್ ಎಂದರೆ ಇನ್‌ಪುಟ್/ಔಟ್‌ಪುಟ್‌ನ್ನು ಬೇರೆ ಕಡೆಗೆ ತಿರುಗಿಸುವುದು.

ಪೈಪ್ ಎಂದರೆ ಒಂದು ಪ್ರೋಗ್ರಾಂನ ಔಟ್‌ಪುಟ್ ಅನ್ನು ಮತ್ತೊಂದರ ಇನ್‌ಪುಟ್‌ಗೆ ಸಂಪರ್ಕಿಸುವುದು.

ಸಿಗ್ನಲ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂಗೆ ಕಳುಹಿಸಲಾಗುವ ಸೂಚನೆ.

ಇಂಟರಪ್ಟ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂ ಚಾಲನೆಯನ್ನು ತಡೆಯುವ ಸಂಕೇತ.

ಪ್ರೊಸೆಸ್ ಎಂದರೆ ಚಾಲನೆಯಲ್ಲಿರುವ ಪ್ರೋಗ್ರಾಂ.

ಪ್ರೊಸೆಸ್ ಐಡಿ ಎಂದರೆ ಪ್ರೊಸೆಸ್‌ಗೆ ನೀಡಲಾದ ಸಂಖ್ಯೆ.

ಪೇರೆಂಟ್ ಪ್ರೊಸೆಸ್ ಎಂದರೆ ಮತ್ತೊಂದು ಪ್ರೊಸೆಸ್ ಅನ್ನು ಪ್ರಾರಂಭಿಸಿದ ಪ್ರೊಸೆಸ್.

ಚೈಲ್ಡ್ ಪ್ರೊಸೆಸ್ ಎಂದರೆ ಮತ್ತೊಂದು ಪ್ರೊಸೆಸ್‌ನಿಂದ ಪ್ರಾರಂಭಿಸಲಾದ ಪ್ರೊಸೆಸ್.

ಡೀಮನ್ ಎಂದರೆ ಹಿನ್ನೆಲೆಯಲ್ಲಿ ಚಲಿಸುವ ಪ್ರೋಗ್ರಾಂ.

ಸರ್ವಿಸ್ ಎಂದರೆ ಹಿನ್ನೆಲೆಯಲ್ಲಿ ಚಲಿಸುವ ಪ್ರೋಗ್ರಾಂ.

ಕ್ರಾಸ್-ಪ್ಲಾಟ್‌ಫಾರ್ಮ್ ಎಂದರೆ ಹಲವು ಪ್ಲಾಟ್‌ಫಾರ್ಮ್‌ಗಳಲ್ಲಿ ಚಲಿಸುವ ಸಾಮರ್ಥ್ಯ.

ನೇಟಿವ್ ಎಂದರೆ ನಿರ್ದಿಷ್ಟ ಪ್ಲಾಟ್‌ಫಾರ್ಮ್‌ಗಾಗಿ ರಚಿಸಲಾದ.

ಪೋರ್ಟೆಬಲ್ ಎಂದರೆ ಸುಲಭವಾಗಿ ಬೇರೆ ಪ್ಲಾಟ್‌ಫಾರ್ಮ್‌ಗೆ ಸಾಗಿಸಬಹುದಾದ.

ಬೈನರಿ ಎಂದರೆ ಯಂತ್ರ ಭಾಷೆಯ ರೂಪ.

ಸೋರ್ಸ್ ಕೋಡ್ ಎಂದರೆ ಮಾನವ ಓದಬಲ್ಲ ಪ್ರೋಗ್ರಾಮಿಂಗ್ ಭಾಷೆಯ ಕೋಡ್.

ಆಪ್ಟಿಮೈಸೇಷನ್ ಎಂದರೆ ಕೋಡ್ ಅನ್ನು ವೇಗವಾಗಿ ಅಥವಾ ಕಡಿಮೆ ಸ್ಥಳದಲ್ಲಿ ಚಲಿಸುವಂತೆ ಮಾಡುವುದು.

ಪ್ರೊಫೈಲಿಂಗ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂನ ಕಾರ್ಯಕ್ಷಮತೆಯನ್ನು ಅಳೆಯುವುದು.

ಬೆಂಚ್‌ಮಾರ್ಕ್ ಎಂದರೆ ಕಾರ್ಯಕ್ಷಮತೆಯನ್ನು ಹೋಲಿಸಲು ಬಳಸುವ ಪರೀಕ್ಷೆ.

ಮೆಮೊರಿ ಲೀಕ್ ಎಂದರೆ ಬಳಸಿದ ಮೆಮೊರಿಯನ್ನು ಬಿಡುಗಡೆ ಮಾಡದಿರುವುದು.

ಗಾರ್ಬೇಜ್ ಕಲೆಕ್ಷನ್ ಎಂದರೆ ಬಳಕೆಯಲ್ಲಿಲ್ಲದ ಮೆಮೊರಿಯನ್ನು ಸ್ವಯಂಚಾಲಿತವಾಗಿ ಬಿಡುಗಡೆ ಮಾಡುವುದು.

ಮ್ಯಾನುಯಲ್ ಮೆಮೊರಿ ಮ್ಯಾನೇಜ್‌ಮೆಂಟ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಮರ್ ಸ್ವತಃ ಮೆಮೊರಿಯನ್ನು ನಿರ್ವಹಿಸುವುದು.

ಪಾಯಿಂಟರ್ ಎಂದರೆ ಮೆಮೊರಿ ವಿಳಾಸವನ್ನು ಸಂಗ್ರಹಿಸುವ ವೇರಿಯಬಲ್.

ರೆಫರೆನ್ಸ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ಗೆ ಪರೋಕ್ಷ ಸಂಪರ್ಕ.

ಡೀರೆಫರೆನ್ಸ್ ಎಂದರೆ ಪಾಯಿಂಟರ್ ಮೂಲಕ ಮೌಲ್ಯವನ್ನು ಪಡೆಯುವುದು.

ಅಡ್ರೆಸ್ ಆಫ್ ಎಂದರೆ ವೇರಿಯಬಲ್‌ನ ವಿಳಾಸವನ್ನು ಪಡೆಯುವುದು.

ಕಾಸ್ಟಿಂಗ್ ಎಂದರೆ ಒಂದು ಪ್ರಕಾರವನ್ನು ಮತ್ತೊಂದಕ್ಕೆ ಪರಿವರ್ತಿಸುವುದು.

ಇಂಪ್ಲಿಸಿಟ್ ಕಾಸ್ಟ್ ಎಂದರೆ ಸ್ವಯಂಚಾಲಿತ ಪರಿವರ್ತನೆ.

ಎಕ್ಸ್‌ಪ್ಲಿಸಿಟ್ ಕಾಸ್ಟ್ ಎಂದರೆ ಸ್ಪಷ್ಟವಾಗಿ ಬರೆದ ಪರಿವರ್ತನೆ.

ಟೆಂಪ್ಲೇಟ್ ಎಂದರೆ ಸಾಮಾನ್ಯ ಕೋಡ್ ರಚನೆಗೆ ಬಳಸುವ ಮಾದರಿ.

ಜೆನೆರಿಕ್ಸ್ ಎಂದರೆ ಪ್ರಕಾರ-ಸ್ವತಂತ್ರ ಕೋಡ್ ರಚನೆ.

ಲ್ಯಾಂಬ್ಡಾ ಎಂದರೆ ಹೆಸರಿಲ್ಲದ ಫಂಕ್ಷನ್.

ಕ್ಲೋಸರ್ ಎಂದರೆ ಸುತ್ತಲಿನ ಸ್ಥಿತಿಯನ್ನು ಸೆರೆಹಿಡಿಯುವ ಫಂಕ್ಷನ್.

ಹೈಯರ್-ಆರ್ಡರ್ ಫಂಕ್ಷನ್ ಎಂದರೆ ಫಂಕ್ಷನ್‌ನ್ನು ಆರ್ಗ್ಯುಮೆಂಟ್ ಅಥವಾ ರಿಟರ್ನ್ ಆಗಿ ಪಡೆಯುವ ಫಂಕ್ಷನ್.

ಪ್ಯೂರ್ ಫಂಕ್ಷನ್ ಎಂದರೆ ಸೈಡ್ ಎಫೆಕ್ಟ್ ಇಲ್ಲದ ಫಂಕ್ಷನ್.

ಇಮ್ಯೂಟೆಬಲ್ ಎಂದರೆ ಬದಲಾಯಿಸಲಾಗದ ಡೇಟಾ.

ಮ್ಯೂಟೆಬಲ್ ಎಂದರೆ ಬದಲಾಯಿಸಬಹುದಾದ ಡೇಟಾ.

ಸೈಡ್ ಎಫೆಕ್ಟ್ ಎಂದರೆ ಫಂಕ್ಷನ್ ಹೊರಗಿನ ಸ್ಥಿತಿಯನ್ನು ಬದಲಾಯಿಸುವುದು.

ಲೇಜಿ ಎವಾಲ್ಯುಯೇಷನ್ ಎಂದರೆ ಅಗತ್ಯವಿದ್ದಾಗ ಮಾತ್ರ ಮೌಲ್ಯವನ್ನು ಲೆಕ್ಕಹಾಕುವುದು.

ಈಗರ್ ಎವಾಲ್ಯುಯೇಷನ್ ಎಂದರೆ ತಕ್ಷಣ ಮೌಲ್ಯವನ್ನು ಲೆಕ್ಕಹಾಕುವುದು.

ಪ್ಯಾಟರ್ನ್ ಮ್ಯಾಚಿಂಗ್ ಎಂದರೆ ರಚನೆಯ ಆಧಾರದಲ್ಲಿ ಡೇಟಾವನ್ನು ಹೊಂದಿಸುವುದು.

ಡೆಸ್ಟ್ರಕ್ಚರಿಂಗ್ ಎಂದರೆ ಸಂಯುಕ್ತ ಡೇಟಾವನ್ನು ಭಾಗಗಳಾಗಿ ವಿಭಜಿಸುವುದು.

ಮ್ಯಾಕ್ರೋ ಎಂದರೆ ಕೋಡ್ ರಚಿಸುವ ಕೋಡ್.

ಹೈಜೀನಿಕ್ ಮ್ಯಾಕ್ರೋ ಎಂದರೆ ಅನಿರೀಕ್ಷಿತ ಹೆಸರು ಘರ್ಷಣೆಯನ್ನು ತಪ್ಪಿಸುವ ಮ್ಯಾಕ್ರೋ.

ಕಂಪೈಲ್-ಟೈಮ್ ಎಂದರೆ ಕಂಪೈಲೇಷನ್ ಸಮಯದಲ್ಲಿ ನಡೆಯುವ ಕಾರ್ಯ.

ರನ್‌ಟೈಮ್ ಎಂದರೆ ಪ್ರೋಗ್ರಾಂ ಚಾಲನೆಯಾಗುವ ಸಮಯ.

ಸ್ಟಾಟಿಕ್ ಟೈಪಿಂಗ್ ಎಂದರೆ ಕಂಪೈಲ್-ಟೈಮ್‌ನಲ್ಲಿ ಪ್ರಕಾರ ಪರಿಶೀಲನೆ.

ಡೈನಾಮಿಕ್ ಟೈಪಿಂಗ್ ಎಂದರೆ ರನ್‌ಟೈಮ್‌ನಲ್ಲಿ ಪ್ರಕಾರ ಪರಿಶೀಲನೆ.

ಸ್ಟ್ರಾಂಗ್ ಟೈಪಿಂಗ್ ಎಂದರೆ ಕಟ್ಟುನಿಟ್ಟಾದ ಪ್ರಕಾರ ನಿಯಮಗಳು.

ವೀಕ್ ಟೈಪಿಂಗ್ ಎಂದರೆ ಸಡಿಲವಾದ ಪ್ರಕಾರ ನಿಯಮಗಳು.

ಟೈಪ್ ಇನ್‌ಫರೆನ್ಸ್ ಎಂದರೆ ಪ್ರಕಾರವನ್ನು ಸ್ವಯಂಚಾಲಿತವಾಗಿ ಊಹಿಸುವುದು.

ಟೈಪ್ ಅನೋಟೇಷನ್ ಎಂದರೆ ಪ್ರಕಾರವನ್ನು ಸ್ಪಷ್ಟವಾಗಿ ಬರೆಯುವುದು.

ಯೂನಿಯನ್ ಟೈಪ್ ಎಂದರೆ ಹಲವು ಪ್ರಕಾರಗಳಲ್ಲಿ ಒಂದಾಗಿರಬಹುದಾದ ಪ್ರಕಾರ.

ಇಂಟರ್‌ಸೆಕ್ಷನ್ ಟೈಪ್ ಎಂದರೆ ಹಲವು ಪ್ರಕಾರಗಳ ಸಂಯೋಜನೆ.

ಟೈಪ್ ಅಲಿಯಾಸ್ ಎಂದರೆ ಪ್ರಕಾರಕ್ಕೆ ಪರ್ಯಾಯ ಹೆಸರು.

ಜೆನೆರಿಕ್ ಕಾನ್‌ಸ್ಟ್ರೈಂಟ್ ಎಂದರೆ ಪ್ರಕಾರ ನಿಯತಾಂಕಕ್ಕೆ ಮಿತಿ.

ಅಸೋಸಿಯೇಟೆಡ್ ಟೈಪ್ ಎಂದರೆ ಟ್ರೇಟ್‌ನಲ್ಲಿ ವ್ಯಾಖ್ಯಾನಿಸಲಾದ ಪ್ರಕಾರ.

ಟ್ರೇಟ್ ಎಂದರೆ ಹಂಚಿಕೊಳ್ಳಬಹುದಾದ ನಡವಳಿಕೆಯ ಒಪ್ಪಂದ.

ಇಂಪ್ಲಿಮೆಂಟೇಷನ್ ಎಂದರೆ ಟ್ರೇಟ್‌ನ ಕಾರ್ಯಗತಗೊಳಿಸುವಿಕೆ.

ಡೈನಾಮಿಕ್ ಡಿಸ್ಪ್ಯಾಚ್ ಎಂದರೆ ರನ್‌ಟೈಮ್‌ನಲ್ಲಿ ಮೆಥಡ್ ಆಯ್ಕೆ.

ಸ್ಟ್ಯಾಟಿಕ್ ಡಿಸ್ಪ್ಯಾಚ್ ಎಂದರೆ ಕಂಪೈಲ್-ಟೈಮ್‌ನಲ್ಲಿ ಮೆಥಡ್ ಆಯ್ಕೆ.

ವರ್ಚುವಲ್ ಟೇಬಲ್ ಎಂದರೆ ಮೆಥಡ್ ವಿಳಾಸಗಳ ಕೋಷ್ಟಕ.

ಮಲ್ಟಿಪಲ್ ಡಿಸ್ಪ್ಯಾಚ್ ಎಂದರೆ ಹಲವು ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳ ಆಧಾರದಲ್ಲಿ ಮೆಥಡ್ ಆಯ್ಕೆ.

ಸಿಂಗಲ್ ಡಿಸ್ಪ್ಯಾಚ್ ಎಂದರೆ ಒಂದು ಆರ್ಗ್ಯುಮೆಂಟ್‌ನ ಆಧಾರದಲ್ಲಿ ಮೆಥಡ್ ಆಯ್ಕೆ.

ಡಬಲ್ ಡಿಸ್ಪ್ಯಾಚ್ ಎಂದರೆ ಎರಡು ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳ ಆಧಾರದಲ್ಲಿ ಮೆಥಡ್ ಆಯ್ಕೆ.

ವಿಸಿಟರ್ ಪ್ಯಾಟರ್ನ್ ಎಂದರೆ ಡೇಟಾ ರಚನೆಯಿಂದ ಕಾರ್ಯವನ್ನು ಬೇರ್ಪಡಿಸುವ ಪ್ಯಾಟರ್ನ್.

ಸಿಂಗಲ್‌ಟನ್ ಎಂದರೆ ಒಂದೇ ಒಂದು ನಿದರ್ಶನವಿರುವ ಕ್ಲಾಸ್.

ಫ್ಯಾಕ್ಟರಿ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್ ರಚನೆಗೆ ಮೀಸಲಾದ ಕ್ಲಾಸ್.

ಅಬ್‌ಸ್ಟ್ರಾಕ್ಟ್ ಫ್ಯಾಕ್ಟರಿ ಎಂದರೆ ಸಂಬಂಧಿತ ಆಬ್ಜೆಕ್ಟ್‌ಗಳ ಕುಟುಂಬಕ್ಕೆ ಫ್ಯಾಕ್ಟರಿ.

ಬಿಲ್ಡರ್ ಎಂದರೆ ಸಂಕೀರ್ಣ ಆಬ್ಜೆಕ್ಟ್ ಅನ್ನು ಹಂತ ಹಂತವಾಗಿ ನಿರ್ಮಿಸುವ ಕ್ಲಾಸ್.

ಪ್ರೊಟೋಟೈಪ್ ಎಂದರೆ ಅಸ್ತಿತ್ವದಲ್ಲಿರುವ ಆಬ್ಜೆಕ್ಟ್‌ನ ಪ್ರತಿಯನ್ನು ಆಧರಿಸಿ ಹೊಸ ಆಬ್ಜೆಕ್ಟ್ ರಚಿಸುವುದು.

ಅಡಾಪ್ಟರ್ ಎಂದರೆ ಒಂದು ಇಂಟರ್‌ಫೇಸ್‌ನ್ನು ಮತ್ತೊಂದಕ್ಕೆ ಪರಿವರ್ತಿಸುವ ಕ್ಲಾಸ್.

ಡೆಕೊರೇಟರ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ಗೆ ಹೊಸ ನಡವಳಿಕೆಯನ್ನು ಸೇರಿಸುವ ಕ್ಲಾಸ್.

ಫೇಸಡ್ ಎಂದರೆ ಸಂಕೀರ್ಣ ವ್ಯವಸ್ಥೆಗೆ ಸರಳ ಇಂಟರ್‌ಫೇಸ್.

ಪ್ರಾಕ್ಸಿ ಎಂದರೆ ಮತ್ತೊಂದು ಆಬ್ಜೆಕ್ಟ್‌ಗೆ ಪ್ರತಿನಿಧಿಯಾಗಿ ಕಾರ್ಯನಿರ್ವಹಿಸುವ ಆಬ್ಜೆಕ್ಟ್.

ಕಾಂಪೋಸಿಟ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ಗಳನ್ನು ಮರದ ರಚನೆಯಲ್ಲಿ ಜೋಡಿಸುವುದು.

ಫ್ಲೈವೇಟ್ ಎಂದರೆ ಹಂಚಿಕೊಳ್ಳಬಹುದಾದ ಆಬ್ಜೆಕ್ಟ್.

ಅಬ್ಸರ್ವರ್ ಎಂದರೆ ಬದಲಾವಣೆಗಳನ್ನು ಗಮನಿಸುವ ಆಬ್ಜೆಕ್ಟ್.

ಸಬ್ಜೆಕ್ಟ್ ಎಂದರೆ ಅಬ್ಸರ್ವರ್‌ಗಳನ್ನು ಹೊಂದಿರುವ ಆಬ್ಜೆಕ್ಟ್.

ಸ್ಟ್ರಾಟಜಿ ಎಂದರೆ ಬದಲಾಯಿಸಬಹುದಾದ ಅಲ್ಗಾರಿದಮ್.

ಟೆಂಪ್ಲೇಟ್ ಮೆಥಡ್ ಎಂದರೆ ಅಲ್ಗಾರಿದಮ್‌ನ ಅಸ್ಥಿಪಂಜರವನ್ನು ವ್ಯಾಖ್ಯಾನಿಸುವ ಮೆಥಡ್.

ಕಮಾಂಡ್ ಎಂದರೆ ಕ್ರಿಯೆಯನ್ನು ಆಬ್ಜೆಕ್ಟ್ ಆಗಿ ಪರಿವರ್ತಿಸುವುದು.

ಇಟರೇಟರ್ ಎಂದರೆ ಸಂಗ್ರಹದ ಮೂಲಕ ಒಂದೊಂದಾಗಿ ಸಾಗುವ ಆಬ್ಜೆಕ್ಟ್.

ಮೆಮೆಂಟೊ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ನ ಸ್ಥಿತಿಯ ಸ್ನ್ಯಾಪ್‌ಶಾಟ್.

ಸ್ಟೇಟ್ ಎಂದರೆ ಆಬ್ಜೆಕ್ಟ್‌ನ ಪ್ರಸ್ತುತ ಸ್ಥಿತಿಯನ್ನು ಪ್ರತಿನಿಧಿಸುವ ಕ್ಲಾಸ್.

ಚೈನ್ ಆಫ್ ರೆಸ್ಪಾನ್ಸಿಬಿಲಿಟಿ ಎಂದರೆ ಹಲವು ಆಬ್ಜೆಕ್ಟ್‌ಗಳಿಗೆ ವಿನಂತಿಯನ್ನು ರವಾನಿಸುವುದು.

ಮೀಡ

• CSS components :

→ Selectors - Select HTML elements to apply styles.

→ properties - Defines the visual characteristics of element.

→ values - Specify the setting of css property.

→ colors - Define text, background, borders, Colors

→ Font - control font family, size, style and weight.

→ Background - Set background colors & images

→ Borders : Define the border style, width and radius.

→ Margin : Controls space outside an elements.

→ Padding : controls the space inside an elements.

→ Layout : controls arrangement of elements using flexbox, grid & positioning.

• HTML VS HTML5

HTML	HTML 5
1. Older version of HTML	1. Latest major version of HTML
2. Limited multimedia support	2. Supports audio & video.
3. Uses external plugins for multimedia	3. No external plugs needed
4. Has fewer semantic tags	4. provides semantic tags.
5. No <canvas> elements	5. Supports <canvas>
6. Limited webstorage	6. Supports local storage.

• CSS VS CSS 3

CSS	CSS 3
1. Earlier version of CSS.	1. Advanced version of CSS
2. Basic styling features	2. provides advanced Styling features.
3. Limited Animations	3. Supports animations.
4. NO flexible layout systems	4. Supports flexbox and grid.

• Design a static webpage using HTML and CSS

→ Pomodoro timer using HTML, CSS

```
<! Doctype html>
<html>
<head>
  <title> pomodro Timer > <titles>
  <style>
    body{
      font-family : Arial :
      background : # f5f5f5 :
      text-align : center :
      padding : 50 px ;
    }
    timer {
      width : 300px ;
      margin : auto ;
      padding : 30 px ;
      background : white ;
      border-radius : 15px ;
      box-shadow : 0 0 10px grey ;
    }
    h1 {
      colour : # e74c3c ;
    }
```

```
.time{
    font-size : 50px ;
    margin : 20px ;
}
button{
    padding : 10px 20px ;
    margin : 5px ;
    border : none ;
    border-radius : 5px ;
    background : # e74c3c ;
    colour : white ;
}
</style>
</head>
<body>
    <div class = "timer">
    <h1> pomodoro timer </h1>
    <div clas = "time"> 25:00 </div>
    <button> start </button>
    <button> Reset </button>
  </div>
</ body>
</html>
```

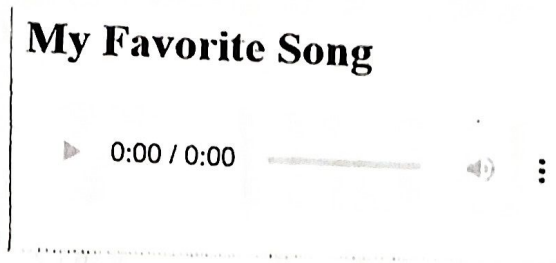
output:



Experiment - 1

1. Design an HTML code using audio tag

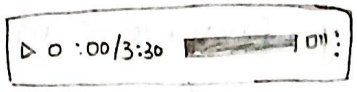
```
<!DOCTYPE html>
<html>
  <head>
    <title> Audio </title>
  </head>
  <body>
    <h2> my favourite song </h2>
    <audio controls>
      <source src = "song.mp3"
        type = "audio/mpeg">
      your browser doesn't support
        audio
      tag
    </audio>
  </body>
</html>
```



• output:

Display an audio player with play, Pause and volume controls.

-> My favourite song



2. Design a HTML code using tabe tag

Code:

```
<table boolder = "1" >
<tr>
 <th> Name <th>
  <th> Age <th>
</tr>
<tr>
  <td> Alice </td>
  <td> 25 </td>
</tr>
<tr>
   <td> Bob </td>
    <td> 30 </td>
</tr>
</table>
```

Output:

Name	Age
Alice	25
Bob	30

3. Design a HTML code using video tags

• code:

```
<! DOCTYPE html>
<html>
  <head>
    <title> video example </title>
  </head>
  <body>
    <h2> my video </h2>
    <video width = "400" controls>
      <source src = "video.mp4"
              type = "video/mp4">
    </video>
  </body>
</html>
```

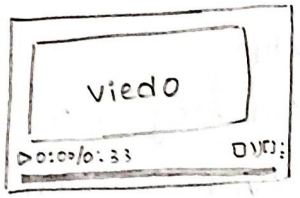
My Video



output:

Display a viedo player with Play, pause volume and full screen controls.

•My video.



4. Design a HTML code using Image tab.

Code:
```
<!DOCTYPE html>
<html lang "en">
<head>
  <title> Image</title>.
</head>
  <body>
    <img src = "https://fruit.com"
    alt = fruit image">

  </body>
</html>
```

output:

fruit image

Experiment - 3

3. Write a program using Javascript (Arrays, Functions)

code:
```
let numbers = (12, 13,14,15,16,19, 24, 34, 67,
              90,99)
Function display (arr){
    console.log("Array elements: ");
    for (let i=0; i<arr.length;i++){
        console.log(arr(i))
    }
}

Function Find sum (arr){
    let sum = 0
    for (let i=0; i<arr.length; ++){
        sum += arr[i]
    }
    return sum;
}
```

JS array.js

```
let numbers = [2, 45, 8, 23, 56, 19, 34, 1, 90];
function displayArray(arr) {
    console.log("Array Elements:");
    for (let i = 0; i < arr.length; i++) {
        console.log arr[i] ;
    }
}
function findSum(arr) {
    let sum = 0;
    for (let i = 0; i < arr.length; i++) {
        sum += arr[i ;
    }
    return sum;
}
displayArray(numbers);
console.log("Sum:", findSum(numbers));
```

Output

```
[Running] node "m:\PYLAB\array.js"
Array Elements:
2
45
8
23
56
19
34
1
90
Sum: 278
```

Experiment - 04

4. Develop dynamic webpage using java script

• Program:

```
<!DOCTYPE html>
<html>
 <head>
    <title> Dynamic webpage </title>
    <style>
       body{
          text-align : center;
          font-family: Arial ;
          margin-top : 100px;
          }
       button{
          padding : 10px x 20px ;
          font-size : 16px ;
          cursor : pointer;
          }
    </style>
 </head>
 <body>
    <h1  id = "heading" > Welcome to
          My webpage </h1>
    <p id = "message"> click the button
       to change this webpage.</p>
    <button onclick = " changepage() "> clickMe</
                        button>
    <script>
       function changepage(){
          document . getElementById ("heading").
```

```
innerHTML = "welcome to JavaScript!";
document.getElementById("heading").
    innerHTML = "welcome" "The webpage
        has been changed dynamically";
document.body.style.backgroundColor = "light
}
</script>
</body>
</html>
```

* output:

Welcome to My Webpage

click the button to change this webpage.

[click me.]

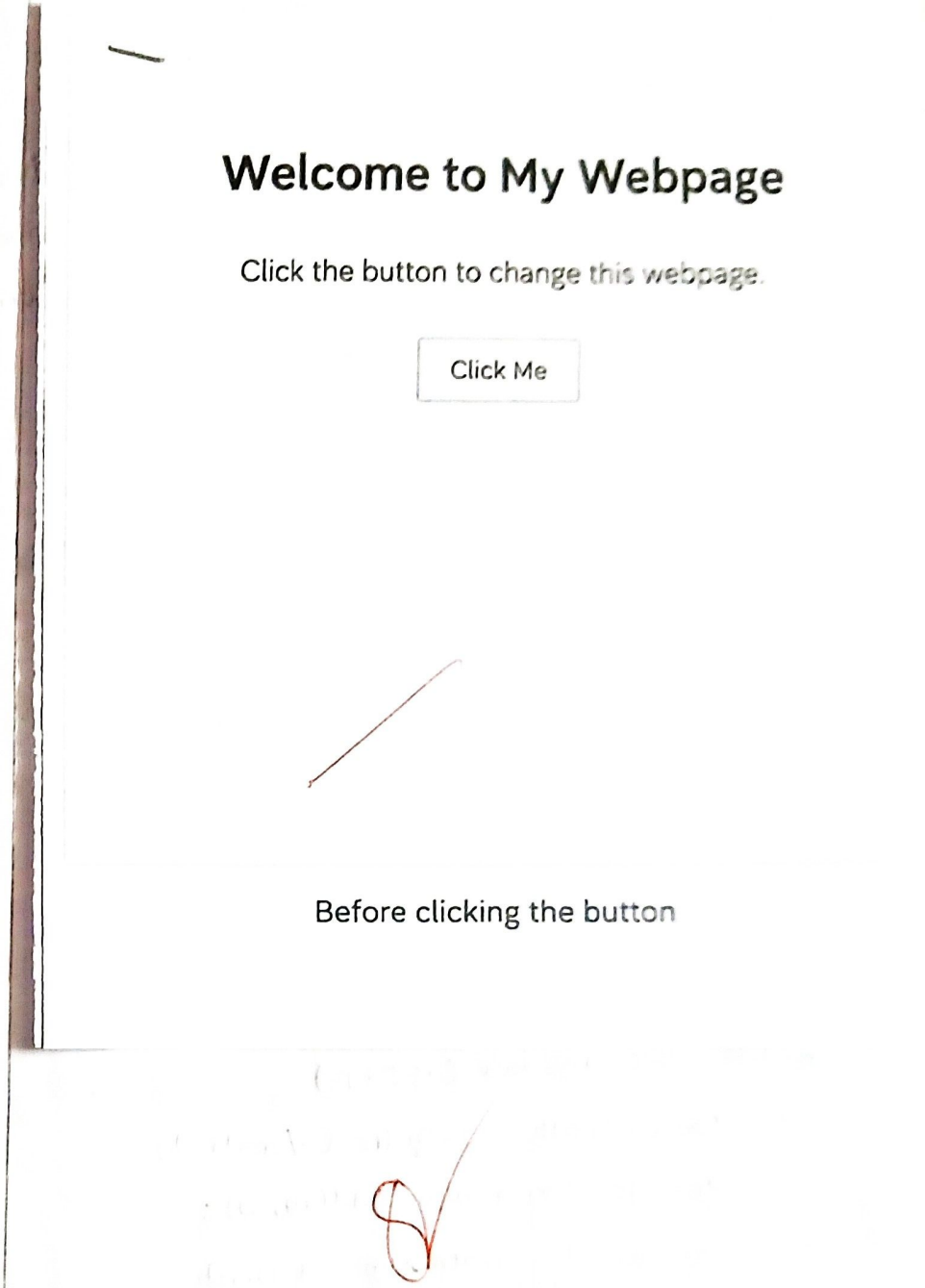
→ when the user clicks "click me", the javas function changepage() is called, then the heading dynamically changes and background also changes.

* After clicking the button:

Welcome to JavaScript

The webpage has been changed dynamically

[click me]



Experiment - 05

Implement Modules in NodeJs.

- In Node.js, you can implement modules using two distinct systems: commonJS(CJS), which is traditional default, and ES modules (ESM), the modern javascript standard.

1. The commonJS Method (default):

commonJS uses module.exports to share code and require() to import it. This system loads modules synchornously.

a. Create the module (math.js):

```
const add = (a,b) => a+b ;
const subtract = (a,b) => a-b;
module.exports = {
        add,
        Subtract
    };
```

b. Use the module (app.js):

```
const math = require ('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

O/P: 15
5

2. The ES modules Method (modern Standard)

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file

a) Create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercalc";
export default calculatorName;
```

b) Use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

O/P: Supercalc
8
2

3) Using Built-in core modules:

Node.js also comes equipped with dozens of built-in core modules (like fs for file system & path for file routing) that don't need installation.

we will load them by module names:

- // common JS style
  const fs = require('fs');
  // ES modules style
  import fs from 'fs';

Here are some of the most commonly used ones;

1. fs = file system operation
2. http = HTTP server and client
3. path = File path utilities
4. os - operating system utilities
5. events - Event handling.
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL passing
10. query string - URL query string handling

eg: Simple HTTP server

```
http.createServer((req,res) => {
    res.writeHead(200,{'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

o/p: Hello World!

Experiment 06

6. Develop mini application using express and Node.js

- simple mini applications using node.js + express.js:

Student Management System

1. Add a student
2. View all students
3. delete a student
4. uses express.js routes
5. Simple HTML interface

- Project Structure:

```
Student.app/
├── server.js
└── public/
    └── index.html
```

Server.js:

```
const express = require("express");
const app = express();
Const PORT = 3000;
let Students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req,res) => {
        res.json(students);
});
```

```
//Add
app post ('/students', (req, res) => {
    const student = {
        id : students.length + 1,
        name : req.body.name,
        course : req.body.course
    };
    students.push(student);
    res.json({message: "Student added
                        successfully.
                Student : student});
});

//Delete
app.delete("/students/:id", (req, res) =>{
    const id = parseInt(req.params.id);
    students = students.filter(student =>
            student.id !== id);
    res.json({
        message: "Student deleted succes
    });
//start server
app listen (PORT, () => {
    console.log('Server running at
                http://localhost:
            ${PORT}');
});
```

- Public/index.html :

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
<head>
<body>
    <h1> Student Management System</h1>
    <input type='text" id ="name"
     Placeholder = "enter student name" >
    <input type ="text" id ="course"
     placeholder= "enter course">
    <button onclick ="add student()">
                Add Student </button>
    <h2> student list </h2>
    <ul id = "studentlist"></ul>
    <script>
        function addstudent(){
            let name = document.getElement
                        By Id("name").
                        value ;
            let course = document.getElement
                        -By Id("course").value;

            fetch("/Students", {
                method:"POST",
                headers:{
                    "const-Type":"
                        "application/json"},
                body: Json.stringfy({
                    name: name,
                    course: course
                })
            })
```

```
    .then(response => response.json())
    .then(data => {
        alert(data.message);
        displayStudents();
    });
}

function displayStudents() {
    fetch("/students").then(response => response.json())
    .then(students => {
        let list = document.getElementById("studentList");
        list.innerHTML = "";
        students.forEach(student => {
            let li = document.createElement("li");
            li.innerHTML = student.id + " - " + student.name + " - " + student.course +
                " <button onclick='deleteStudent(" + student.id + ")'>Delete</button>";
            list.appendChild(li);
        });
    });
}

function deleteStudent(id) {
    fetch("/students/" + id, { method: "DELETE" })
    .then(response => response.json())
    .then(data => {
        alert(data.message);
        displayStudents();
    });
}

displayStudents();
</script>
</body>
</html>
```

O/P:

Student management System

Enter Student name

enter course

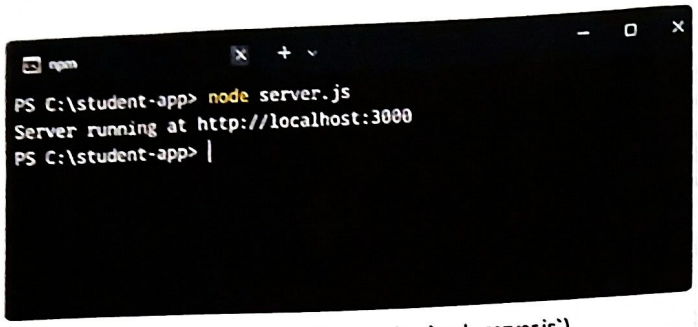
Add Student

Student list

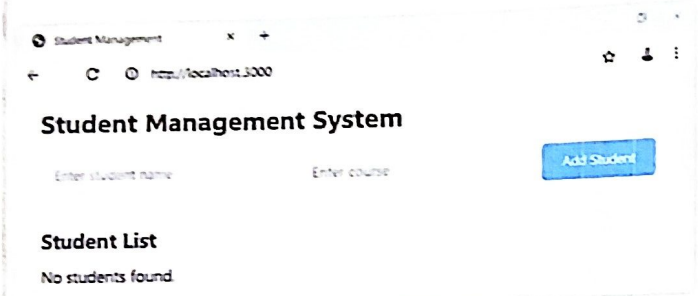
1. Tejaswini - CSM delete
2. Sita - CSE delete

- The Express.js server is successfully started at http://localhost:3000
- The browser displays the Student management System with fields.

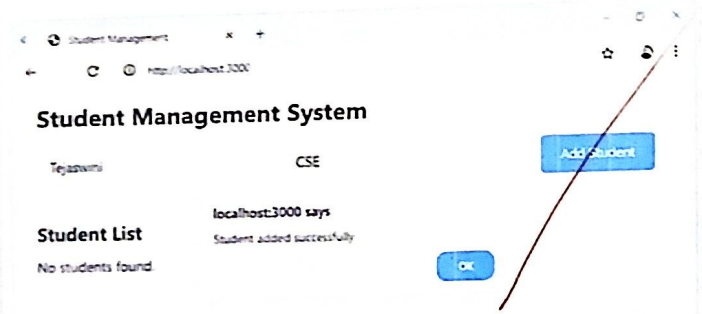
Output:



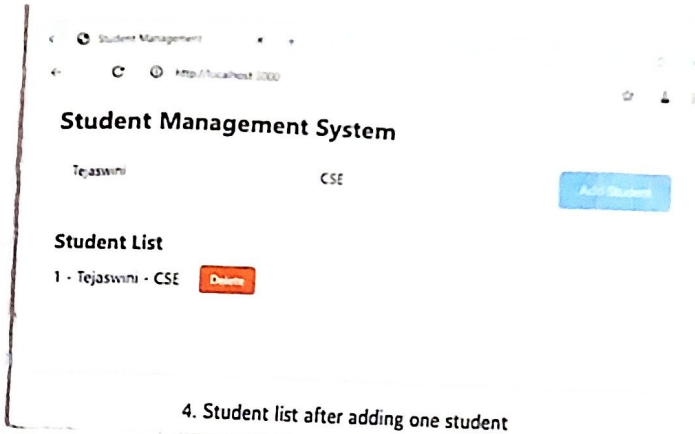
1. Terminal Output (after running `node server.js`)



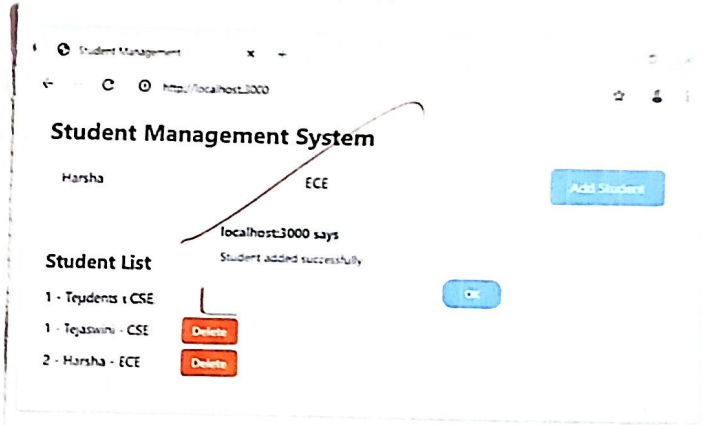
2. Initial Output in Browser (no students)



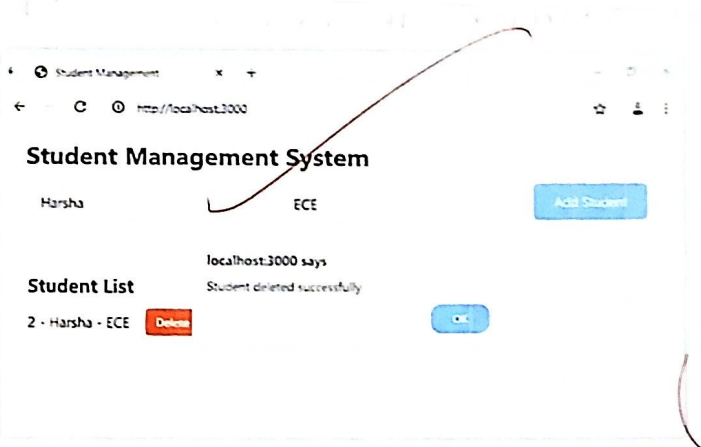
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

1. Create a functional component & class component using react

- Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a web page.

- Functional component:

A functional component is a javascript functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a web page.

- Functional component:

A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the webpage dynamically.

- Program:

```
import React, {useState} from
'react';
function studentfunction(){
    const [name, SetName] = usestate("Sruthi");
    const [course, setcourse] = usestate("CSM");
    const [count, setcount] = usestate(0);
    const changeName = () => {
        SetName("Teju");
    };
    const changecourse = () => {
        SetCourse("CSD");
    };
    const incrementCount = () => {
        SetCount(Count+1);
    };
    return (
        <div style = {{
            border : "8Px solid blue",
            padding : "20px",
            margin : "20px",
            borderRadius : "10px",
            textAlign : "center"
        }}>
        <h1> studentDetails </h1>
        <h2> Functional component </h2>
        <P> <b> Name : </b> {name} </P>
        <P> <b> course : </b> {course} </P>
        <button onclick ={changeName}>
            changeName </button>
        <button onclick ={changecourse}> change
            Course </button>
        <button onclick = {incrementcount}>
            Increment </button>
        </div>
    );
```

*output: when the user clicks the buttons, the name, course and count change dynamically without reloading the web page.

- class component:

A class component is an ES6 Javascript class that extends React. component. It uses the render() method to return JSX and manages state using this.state and this.setState().

- program:

```
import React, {Component} from
'react';
class StudentClass extends Component{
    constructor(props) {
        super(props);
        this.state = {
            name = "Mohana",
            course = "computer science",
            count = 0
        };
    }
    changeName = () => {
        this.setState ({
            name : "Mohana"
        });
    };
    changeCourse = () => {
        this.setState ({
            course : "CSH"
        });
    };
    incrementCount = () => {
        this.setState(prevState) => ({
            count: prevState.count+1
        }));
    };
    render() {
        return (
            <div style = {{
                border : "2px solid green",
                padding : "20px",
                margin : "20px",
                borderRadius : "10px",
                textAlign : "center"
            }}>
            <h1>Student Details </h1>
            <h2> class Component </h2>
            <p> <b> Name : </b> {this.state.name}</p>
            <p> <b> Course: </b> {this.state.course}</p>
            <p> <b> ButtonClicks: </b> {this.state.count}
            </p>
            <button onClick = {this.changeName}>
                changeName </button>
            <button onClick={this.changeCourse}>
                Change course </button>
            <button onClick = {this.incrementCount}> increment
                </button>
        </div>
```

Experiment-08

8. Implement http methods using JSON server

- Aim: To Implement HTTP method such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update and Delete) operations on students records.

- Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server

- GET - Retrieves data from the server.
- POST - Adds New data to the server.
- PUT - Replaces an existing record
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server.

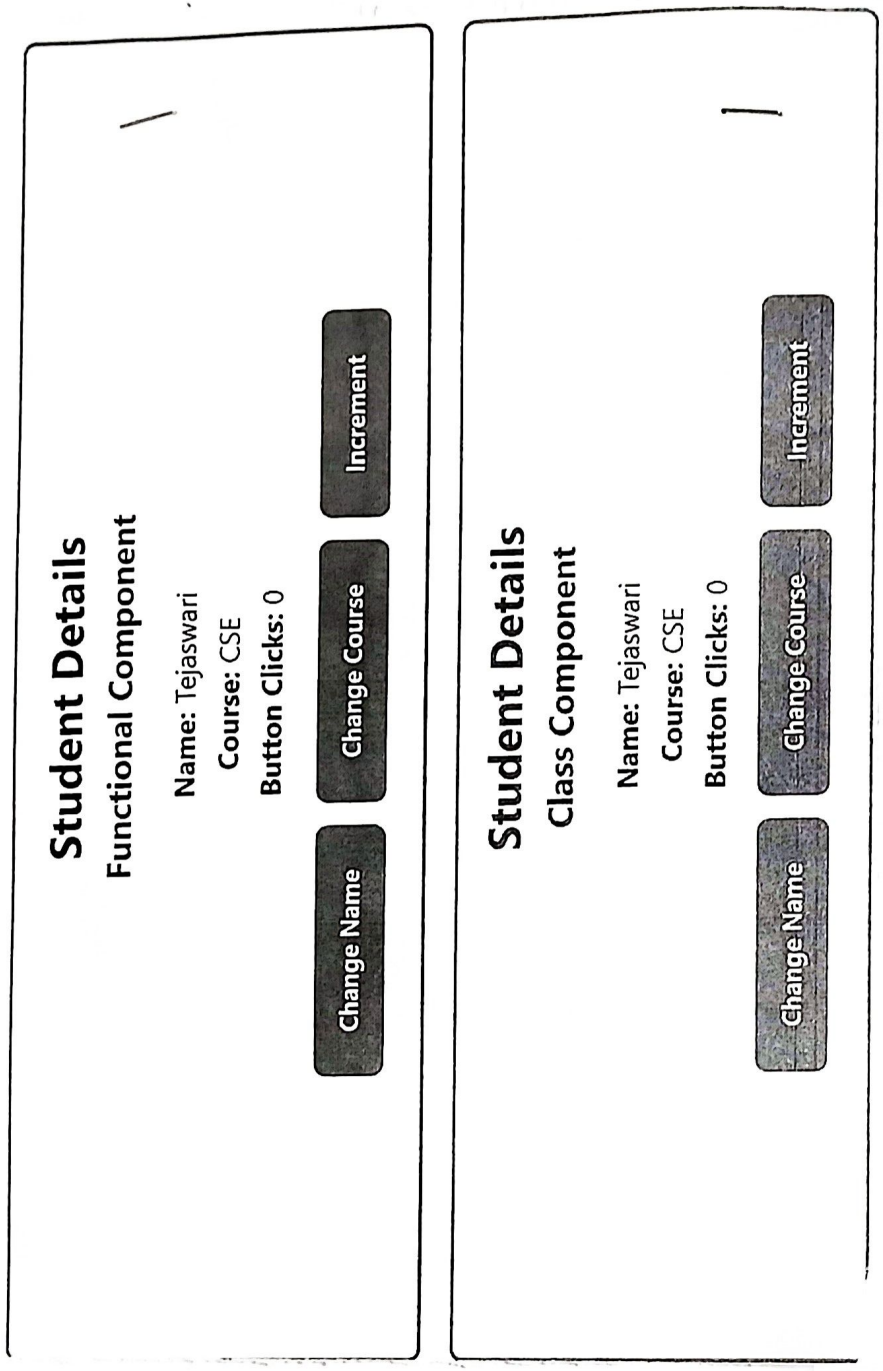
a. Create a JSON server:

- Step 1: Initialize the project

Open the vs code terminal and execute the following commands:

mkdir JSON server
Cd JSONServer.

- Output:



npm init -y
npm install json server.

- Step 2: create db.json.

Create a file named db.json and add the following student data.

```
{
  "students" : [
    {
      "id" : "1";
      "name" : "mohana",
      "course" : "CSM",
      "marks" : 90
    },
    {
      "id" : "2",
      "name" : "sruthi",
      "course" : "CSM",
      "Marks" : 90
    },
    {
      "id" : "3"
      "name" : "harshini",
      "course" : "IT
      "Marks" : 88
    }
```

- Step 3: start the JSON Server

For the current JSON server version, execute:

npx json.server db.json.

The server start at:

http://localhost:3000

The student records can be accessed using:

http://local host:3000/students.

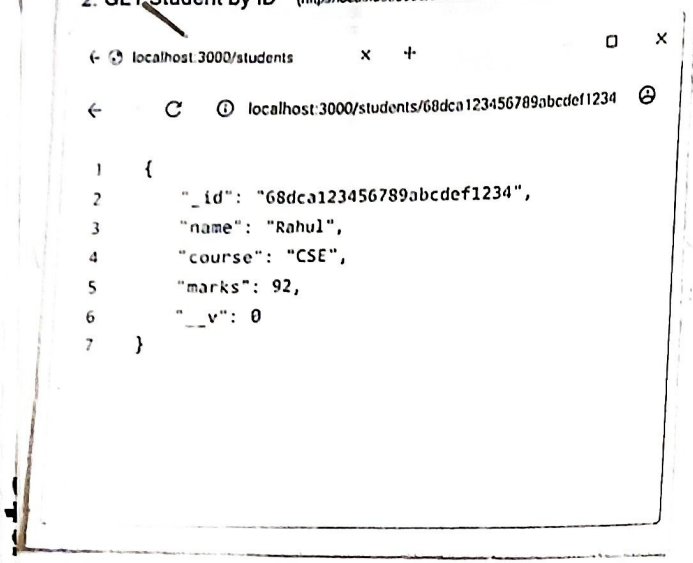
2. Implementation of HTTP method

a. GET Method: The GET method is used to retrieve student records from JSON server.

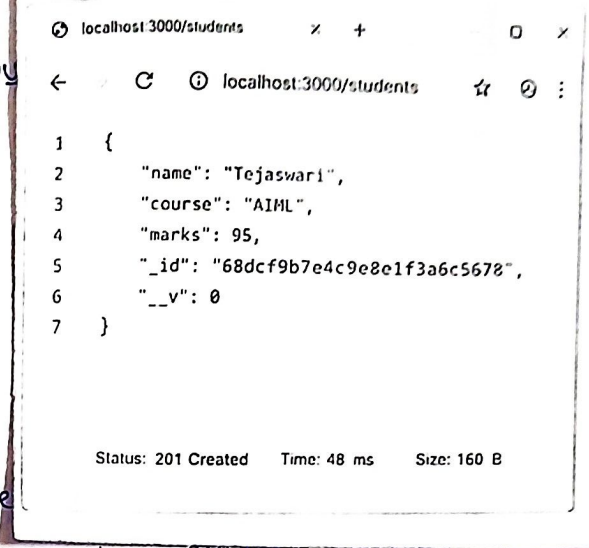
Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json()).
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



3. POST (Create Student) - Using Postman

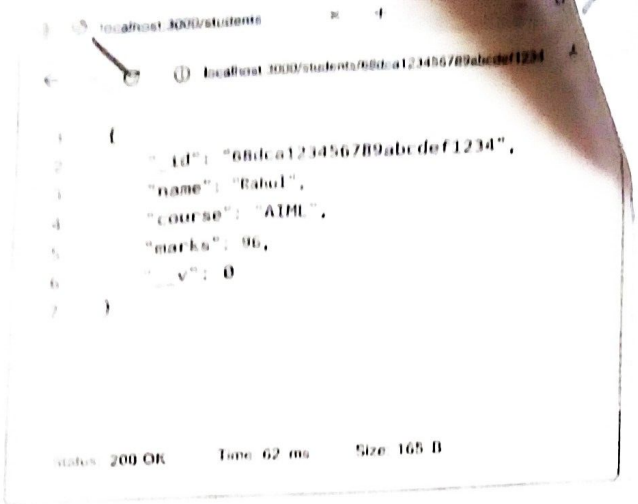


B. POST Method: The post method is used to insert a new student record into the server.

```
code: fetch("http://localhost:3000/students",{
        method: 'POST',
        header: {
          'content-Type': 'application/json'
        },
        body: JSON.stringify({
            name: 'Rahul',
            course: 'CSE',
            marks: 92
        })
      })
    • then (response => response.json())
    • then (data => {
          console.log(data);
```

C. PUT Method: To replace an existing student record with updated data.

```
code: fetch('http://localhost:3000/students/1',{
        method: 'PUT'
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            name: 'Mohana',
            course: 'AIML',
            Marks: 95
        })
      })
    • then (response => response.json())
    • then (data => {
          console.log(data);
      })
    • catch (error => {
          console.log(error);
      });
```



D. PAT

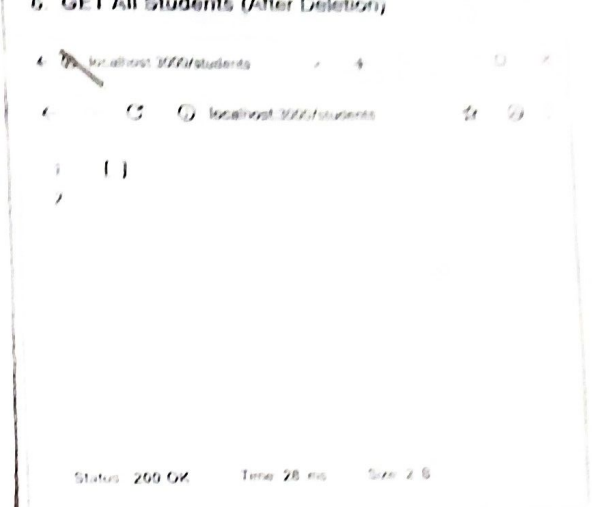
is used to

Update only selected fields of an existing Student record.

- code:

```
fetch('http://localhost:3000/students/2',{
    method: 'PATCH',
    header: {
        'content-Type':'application/json'
    },
    body: JSON.stringify({
        marks: 96
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

- output:



E. DELETE

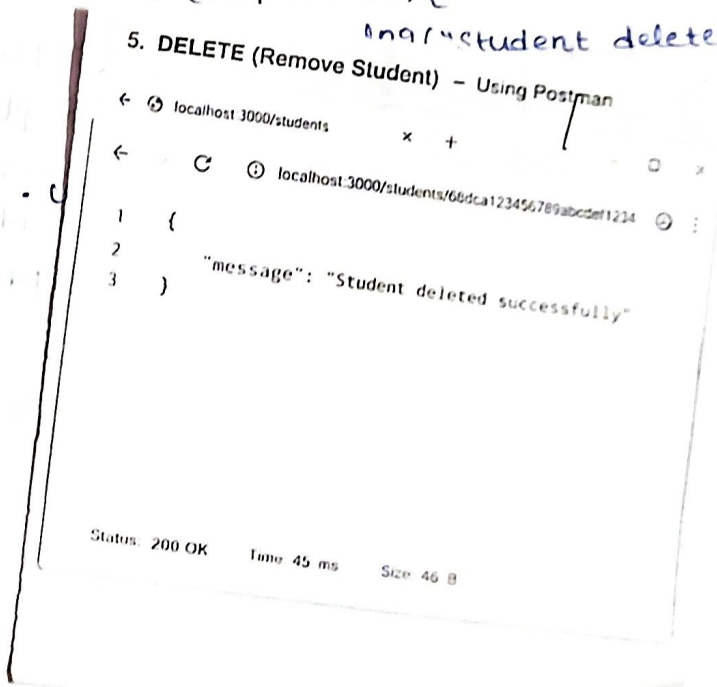
ed to

remove

the server.

- code:

```
fetch("http://localhost:3000/student/3",{
    method: 'DELETE'
})
.then(response => {
    if(response.ok){
        return "student deleted
```



- output:

- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining Stu

- Out

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

localhost:3000/students


```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```


Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were succesfully implemented using JSON Server. The CRUD operations were Performed on student records. the data was reteriered, inserted, updated and deleted using HTTP requests.

Experiment-9

9. Develop CRUD operations using MongoDB

- Aim: To develop a webapplication using Node.js, express.js and MongoDB to perform CRUD operations. (Create, read, update, delete) on Student records.

- Theory: MongoDB is a NoSQL database that Stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database.

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and project setup:

```
mkdir MongoDB_CRUD
cd MongoDB_CRUD
npm.init-y
npm install express mongoose
```

2. Program: server.js

```
const express=require('express');
const mongoose require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
```

```
mogoose.connect('mongodb://127.0.0.1:27017/
                                    College')
.then() => console.log("MongoDB Connected")
.catch(err => console.log(err));

//create student
const StudentSchema = new
mongoose.schema({
        name :{
            type :string
            required : true
        }
        Course :{
            type : string,
            required : true.
        }
        marks : {
            type : Number,
            required : true.
        }
    });
//create Student Model
const Student :
        mongoose.model('Student', Student
                                    Schema);
//Create Add.
app.post('/students', async(req, res)=>{
    try{
        Const Student = new Student(req.body
        const SavedStudent = await Student
                                    save();
    res.status(201).json(SavedStudent);
    } catch(error){
    res.status(400).json({message: error.
                                    message})
    }
});
//READ : Get all students.
app.get('/students', async(req, res) => {
    try {
        const students = await
        Student.find();
        res.status(500).json({message :
                                    error.message});
    }
});
//READ : Get student by ID
app.get('/students/:id', async(req,res) => {
    try {
        const student = await
        Student.findById(req.params.id);
        if(!student){
            return res.status(404).json({
                message: "student not found"
            });
        }
        res.status(200).json(student)
    } catch(error) {
        res.status(400).json({message:
        error.message});
    }
});
```

```
//Update
app.put('/students/:id', async(req,res) => {
    try {
        const student = await
    student.findByIdAndUpdate(
            req.params.id,
            req.body,
            {new: try, runValidators: true}
        );
    if (!student) {
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json(student);
    }catch(error){
        res.status(400).json({message:
            error.message});
    }
});

//DELETE
app.delete('/students/:id', async(req,res)=>
    try{
        const student = await student.
            findByIdAndDelete(
            req.params.id
        );
    if(!student){
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json({
        message: "student delete successfully"
    });
    }catch(error){
        res.status(400).json({message:
            error.message});
    }
});
```

*output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

1. Terminal Output

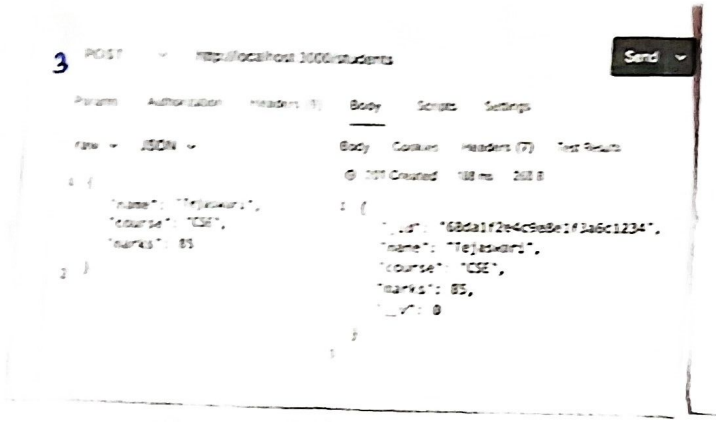
localhost:3000

localhost:3000

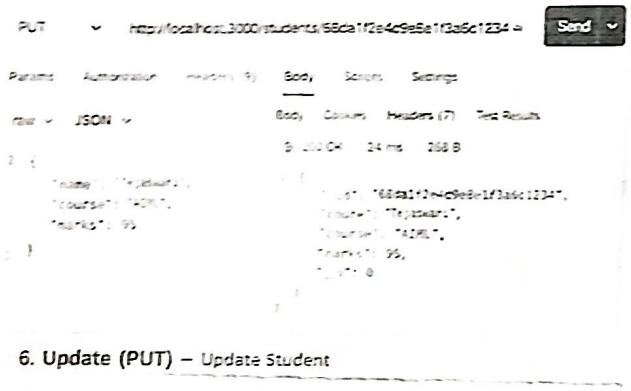
MongoDB CRUD API

Server is running...

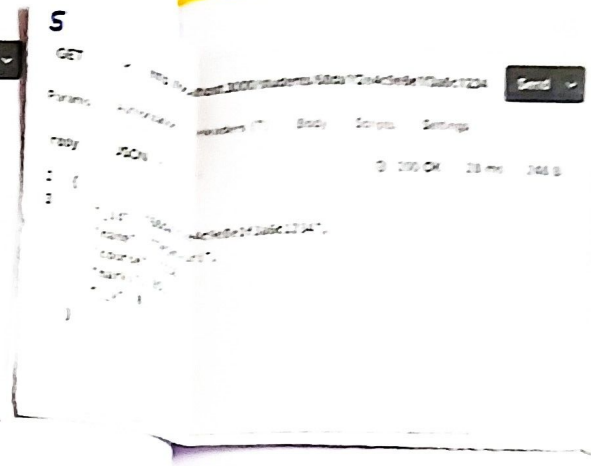
2. Chrome Browser (Home Page)



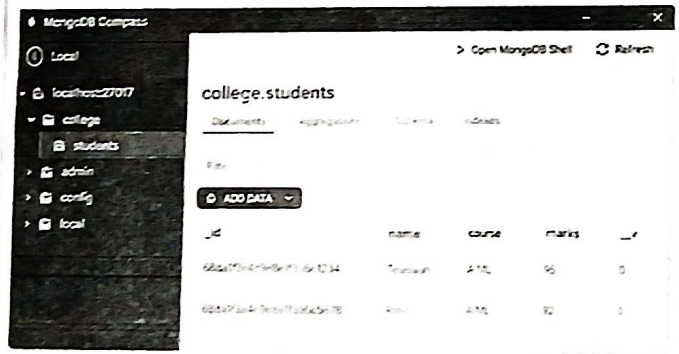
3. Create (POST) – Add Student



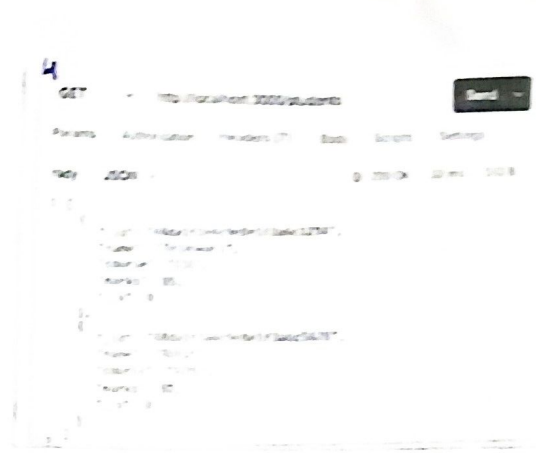
6. Update (PUT) – Update Student



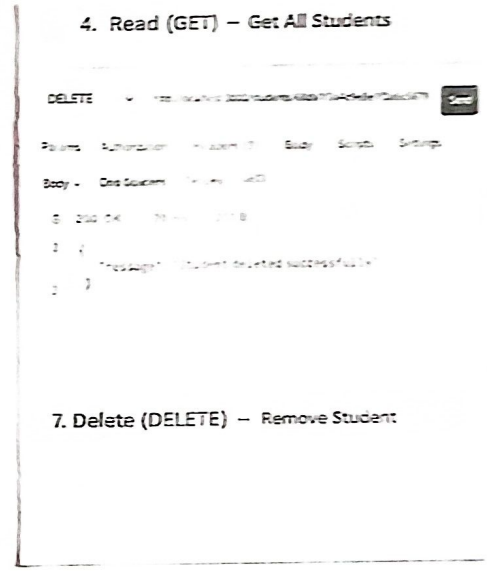
5. Read (GET) – Get Student by ID



8. MongoDB Compass – Database View



4. Read (GET) – Get All Students



7. Delete (DELETE) – Remove Student